

Digital Image Processing — Assignment 1 Report

Student Name: Vishesh Singhal Roll Number: 2024117001

September 9, 2026

System Environment

- **CPU:** 13th Gen Intel® Core™ i5-13420H $\times$ 12
- **RAM:** 16 GB DDR4
- **NumPy Version:** 2.5.2
- **OS:** Fedora Linux 42

1 Problem 1: Convolution

1.1 1.1 Correctness

Four implementations were evaluated with `mode="same"` and zero-padding on a 128×128 crop of `p1/base_2048.png` against `scipy.signal.convolve2d` inbuilt convolution. Table 1.1 gives the max error observed by doing this.

Table 1: Max absolute error relative to `scipy.signal.convolve2d` ($k = 7$ Gaussian).

Implementation	Max Absolute Error
<code>conv2d_loops</code>	$1.990e^{13}$
<code>conv2d_taps</code>	$1.990e^{13}$
<code>conv2d_im2col</code>	$1.137e^{13}$
<code>conv2d_fft</code>	$2.247e^{13}$

Asymmetric Kernel Verification: Evaluating an asymmetric kernel $K_{\text{asym}} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$:

- Error vs `scipy.signal.convolve2d`: **0.00** (confirms true convolution).
- Difference vs `scipy.signal.correlate2d`: $3.242e^3$ (non-zero; confirms distinction from correlation).

Loop Timing and Extrapolation:

- Measured execution time (128×128 , $k = 7$): **0.5650** seconds.
- Scaling factor: $\left(\frac{2048}{128}\right)^2 \times \left(\frac{15}{7}\right)^2 = 256 \times 4.5918 \approx 1175.51$.
- Extrapolated execution time (2048×2048 , $k = 15$): **664.14** seconds ($\approx$ **11.07** minutes).

Reason full-size was not run: Four nested interpreted Python loops have high per-iteration bytecode evaluation overhead. Processing $(2048^2) \times (15^2) \approx 9.44 \times 10^8$ loop iterations without compiler acceleration would take hours to execute.

1.2 1.2 Kernel Rank

Singular Value Decomposition factors a kernel as $K = U\Sigma V^T$. The numerical rank is determined by counting singular values exceeding threshold $\tau = 10^{-3} \cdot \sigma_1$.

- Output singular value spectra plot: see `output/p1/rank_spectra.png`.
- **Significance of threshold τ :** Due to floating point roundoff error, a threshold is required to actually find true singular value not just computational roundoff.

Table 2: Numerical rank and leading singular values of the kernel bank at $k = 15$.

Kernel	Numerical Rank (r)	Leading Singular Value (σ_1)
box	1	$6.67e^{-02}$
gaussian	1	$1.13e^{-01}$
sobel3	1	3.46
log	2	$9.68e^{-01}$
log_dc_removed	3	$9.64e^{-01}$
motion_0deg	1	$2.58e^{-01}$
motion_45deg	15	$6.67e^{-02}$
disk	5	$7.11e^{-02}$
random	14	$4.12e^{-02}$

(a) Derivation of Raw LoG Rank: The Laplacian of Gaussian kernel is defined as:

$$\text{LoG}(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (1)$$

Expanding the polynomial term:

$$\text{LoG}(x, y) = \left(\frac{x^2 - \sigma^2}{\sigma^4}\right) e^{-\frac{x^2}{2\sigma^2}} e^{-\frac{y^2}{2\sigma^2}} + e^{-\frac{x^2}{2\sigma^2}} \left(\frac{y^2 - \sigma^2}{\sigma^4}\right) e^{-\frac{y^2}{2\sigma^2}} \quad (2)$$

Defining 1-D functions $f_1(x) = \frac{x^2 - \sigma^2}{\sigma^4} e^{-\frac{x^2}{2\sigma^2}}$, $g_1(y) = e^{-\frac{y^2}{2\sigma^2}}$, and $f_2(x) = e^{-\frac{x^2}{2\sigma^2}}$, $g_2(y) = \frac{y^2 - \sigma^2}{\sigma^4} e^{-\frac{y^2}{2\sigma^2}}$, the 2-D continuous LoG is the sum of two separable outer products:

$$\text{LoG}(x, y) = f_1(x)g_1(y) + f_2(x)g_2(y) \implies K_{\text{LoG}} = \mathbf{u}_1 \mathbf{v}_1^T + \mathbf{u}_2 \mathbf{v}_2^T \quad (3)$$

Because the vectors $\mathbf{u}_1, \mathbf{u}_2$ are linearly independent, the matrix has rank exactly 2.

(b) Rank Increase of Mean-Subtracted LoG: The DC-removed LoG kernel is formed as:

$$K_{\text{dc}} = K_{\text{LoG}} - \mu \mathbf{1}_{k \times k} = K_{\text{LoG}} - \mu \mathbf{1} \mathbf{1}^T \quad (4)$$

where $\mathbf{1}$ is the all-ones vector. By matrix rank subadditivity:

$$\text{rank}(A + B) \leq \text{rank}(A) + \text{rank}(B) \implies \text{rank}(K_{\text{dc}}) \leq \text{rank}(K_{\text{LoG}}) + \text{rank}(-\mu \mathbf{1} \mathbf{1}^T) = 2 + 1 = 3 \quad (5)$$

Equality holds because $\mathbf{1} \notin \text{span}\{\mathbf{u}_1, \mathbf{u}_2\}$. Since $\mathbf{1}$ is linearly independent of the original column space it was able to raise the rank from 2 to 3. However, this won't be the case for rank 1 constant matrix, as rank-1 perturbation will lie in the span of the original constant matrix. Therefore, this would not necessary increase the rank.

(c) 0° Motion vs 45° Motion Blur Rank: A horizontal motion blur kernel is non-zero along a single center row: $K_{0^\circ} = \mathbf{e}_{\lfloor k/2 \rfloor} (\frac{1}{k} \mathbf{1}^T)$, which has only one linearly independent row, hence rank = 1. The diagonal motion blur kernel is a scaled identity matrix: $K_{45^\circ} = \frac{1}{k} I_{k \times k}$. The identity matrix has k orthogonal linearly independent basis vectors, giving full rank k . While both represent 1D linear physical motion, they both have different span and rank.

1.3 Separability and Low Rank Approximation

Truncated SVD approximates a kernel as $K_r = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$, filtering images through r successive separable passes.

- Output PSNR vs rank curve: see `/p1/psnr_vs_rank.png`.
- **Disk vs Motion- 45° Behavior:** The disk kernel has smooth boundaries whose 2-D energy concentrates heavily within the first few dominant singular values (σ_i drops sharply), allowing $r = 3$ terms to recover most of the details (> 35 dB). Motion- 45° is $K = \frac{1}{k} I$, with singular values all identical: $\sigma_1 = \sigma_2 = \dots = \sigma_k = \frac{1}{k}$. Because there is no singular value decay, no low-rank approximate representation exists, requiring all k terms for reconstruction.

1.4 Runtime Analysis

- Output log-log sweeps: see `p1/runtime_vs_k.png` and `p1/runtime_vs_n.png`.

im2col Patch Matrix Memory Derivation: $N \times N$ image yields N^2 sliding patches of dimension $k \times k$. The unrolled patch matrix has shape (N^2, k^2) . With 64-bit floating point precision (8 bytes/element), peak memory is:

$$M_{\text{peak}} = N^2 \cdot k^2 \cdot 8 \text{ bytes} = \frac{8N^2k^2}{1024^2} \text{ MB} \quad (6)$$

Table 3: Peak im2col patch-matrix memory requirements at $k = 15$.

N	Matrix Shape	Memory (MB)	Memory (GiB)
128	$16,384 \times 225$	28.12 MB	0.0275 GiB
256	$65,536 \times 225$	112.50 MB	0.1099 GiB
512	$262,144 \times 225$	450.00 MB	0.4395 GiB
1024	$1,048,576 \times 225$	1800.00 MB	1.7578 GiB
2048	$4,194,304 \times 225$	7200.00 MB	7.0312 GiB

Behavior at $N = 2048, k = 15$: Allocating 7.03 GiB for a single intermediate matrix causes high memory pressure. It triggers virtual memory paging (swap thrashing) or out-of-memory kernel termination.

(a) **Tap Loop Scaling vs k^2 :** Theoretical scaling of the tap loop is $\mathcal{O}(N^2k^2)$. The measured empirical slope is **1.76** (< 2). The missing exponent is absorbed by vectorized array broadcasting and cache memory bandwidth limits: for small k , memory allocation and stride indexing dominate arithmetic operations.

(b) **im2col vs Tap Loop Performance:** On this system, `im2col` was Slower than `conv2d_taps`. While `im2col` leverages numpy (cpp compiled) matrix multiplication, building and copying the massive patch matrix for a single 2-D channel incurs memory bandwidth overhead that counteracts that efficiency. `im2col` benefits multi-channel CNN layers where the patch matrix is reused across dozens of filter channels, reducing memory overhead relatively.

(c) **Separable Convolution Speedup Discrepancy:** Theoretical speedup for a separable kernel is $\frac{k^2}{2k} = \frac{k}{2} = 7.5\times$ at $k = 15$.

- Measured speedup at 512×512 : **3.81** $\times$.
- Measured speedup at 2048×2048 : **7.69** $\times$.

Discrepancy Explanation: At $N = 512$, intermediate vertical convolution arrays fit inside CPU L2/L3 cache hierarchies so we don't see that level of speedup as memory is not a bottleneck. Whereas, at $N = 2048$, buffer exceed CPU cache capacities, causing memory writes and reads overhead. Here is where we see the true theoretical gain.

2 Problem 2: Bit-Plane Slicing and Steganography

2.1 2.1 Decomposition and Reconstruction

- Bit-plane decomposition display: see `p2/textured_bit_planes.png`.
- Reconstruction PSNR curve: see `p2/psnr_vs_n.png`.
- Linear ramp vs Gray-coded decomposition: see `p2/ramp_vs_gray_decomposition.png`.

Table 4: Reconstruction PSNR keeping the top n bit planes on `cover_textured.png`.

Top n Planes Kept	PSNR (dB)	Incremental Gain (dB)
1	11.89 db	—
2	16.92 db	+5.08 db
3	23.00 db	+6.08 db
4	29.23 db	+6.23 db
5	35.71 db	+6.48 db
6	42.69 db	+6.98 db
7	51.14 db	+8.45 db
8	∞	—

Constant PSNR Gain Explanation: The measured slope across finite planes is 6.50 dB/plane. When truncating plane b , quantization error per pixel is uniformly distributed over $[-2^b/2, 2^b/2]$. Retaining an additional bit plane halves the quantization step size q , reducing error variance by a factor of 4: $\text{MSE}_n \approx \frac{1}{4}\text{MSE}_{n-1}$. By the definition of PSNR:

$$\Delta\text{PSNR} = 10 \log_{10} \left(\frac{\text{MSE}_{n-1}}{\text{MSE}_n} \right) \approx 10 \log_{10}(4) \approx 6.02 \text{ dB/bit} \quad (7)$$

Ramp Artifacts: Standard Binary vs Gray Code: A standard binary ramp produces high-frequency toggling artifacts across lower bit planes at boundaries (e.g., $127 = 01111111_2 \rightarrow 128 = 10000000_2$ flips all 8 bits simultaneously). Gray code transforms values via $g = v \oplus (v \gg 1)$, ensuring that any two adjacent integers differ by exactly one bit. This removes Hamming cliffs and eliminates high-frequency boundary patterns in the lower bit planes.

2.2 2.2 Payload Recovery and Steganalysis

- Header recovered: Sync byte = `0b10101010` (170), Height $H = 64$, Width $W = 64$.
- Extracted binary image output: see `p2/recovered_payload.png`.
- Stego PSNR relative to clean cover: 75.24 dB.
- Difference map output: see `output/p2/difference_map.png`.
- **Visual Detectability:** The payload is invisible to human inspection in the spatial image domain because modifying bit plane 0 shifts pixel values by at most ± 1 level (PSNR > 50 dB).

Comparison: Textured vs Smooth Cover: Embedding the payload into `cover_smooth.png` produces PSNR of 75.28 dB, virtually identical to the textured cover. However, spatial gradient residual filtering $\nabla_x I = |I(x+1, y) - I(x, y)|$ reveals clear steganalysis evidence (see `p2/textured_vs_smooth_evidence.png`):

- In `cover_smooth.png`, the original image has near-zero gradient energy; embedding the payload introduces high-frequency variance that clearly delineates the active payload rows.
- In `cover_textured.png`, intrinsic image gradients mask the LSB perturbations.

2.3 2.3 Robust Steganography Channel Design

2.3.1 Robust Steganography Scheme (Encoding, Distortion Budget, and Decoding)

- **Encoding:** The 1024×1024 image is divided into a grid of 128 non-overlapping blocks. Each secret bit $b_k \in \{0, 1\}$ is assigned to one full block. The entire block pixel intensity was increased by +2 if secret bit is 1 and by -2 if secret bit is 0. Because pixels are shifted by ± 2 , our theoretical squared error is $\text{MSE} = (\pm 2)^2 = 4.0$. This achieves:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{4} \right) \approx 42.10 \text{ dB} \geq 40 \text{ dB} \quad (8)$$

Within our budget.

- **Decoder Type (Cover-Assisted):** The decoder subtracts the clean cover from the stego image and checks each block's average difference: positive for bit 1, negative for bit 0. Decoding blindly without the cover would instead require comparing adjacent blocks or modulating DCT coefficients.

Design Trade-Off Justification: On the capacity/robustness/imperceptibility triangle, we chose low payload capacity (128 bits) to optimize robustness and imperceptibility. By assigning 8,192 pixels to each bit, spatial averaging reduces additive Gaussian noise variance by $\sqrt{8192} \approx 90.5\times$. Similarly, low-frequency block shifts survive 8×8 block DCT quantization at JPEG quality 75, satisfying zero-error decoding requirements while keeping PSNR above 40 dB.

Table 5: Channel evaluation results for naive LSB vs block-mean robust scheme.

Channel Condition	Naive LSB BER	Robust Scheme BER
Gaussian $\sigma = 0$	0	0
Gaussian $\sigma = 1$	0.5625	0
Gaussian $\sigma = 2$	0.5781	0
Gaussian $\sigma = 5$	0.5625	0
Gaussian $\sigma = 10$	0.5	0
Gaussian $\sigma = 20$	0.5156	0.0234
JPEG ($Q = 75$)	0.5391	0

3 Problem 3: Piecewise Linear Transforms

3.1 3.1 Recovering Known Transforms (Pairs A and B)

- Fitted curves and scatter overlays: see `p3/scatter_A.png` and `p3/scatter_B.png`.

Table 6: Recovered segment parameters and re-application errors for Transforms A and B.

Case	Breakpoints	Slopes	Intecepts	Max Error	Mean Abs Error
Pair A	[0, 12, 60, 255]	[0, 3.96, 0.33]	[0, -48.25, 169.67]	0	0
Pair B	[0, 70, 110, 150, 255]	[0.28, 5.37, -5.37, -0.2]	[-0.4, -356.5, 826, 48]	0	0

Invertibility of Transform B: Transform B is non-injective (multiple input gray levels map to identical output levels in the sliced region), which prevents inversion from the output image alone despite the forward mapping being uniquely recoverable from the input-output pair.

3.2 3.2 Model Selection and Quantization Floor (Pair C)

To prevent overfitting in piecewise linear regression, segment count k is selected using the Bayesian Information Criterion (BIC) with an explicit integer quantization noise floor:

$$\text{BIC}(k) = N \ln(\max(\text{MSE}(k), \sigma_q^2)) + (3k - 1) \ln N \quad (9)$$

where $\sigma_q^2 = \frac{1}{12} \approx 0.0833$, the variance of continuous values rounded to integer 8-bit levels.

- Output scatter plot with fitted segments: see `p3/scatter_C.png`.
- Recovered segments count for Pair C: $k = 4$.
- Re-application error on Pair C: Max Absolute Error = 1 level, Mean Absolute Error = 0.0007 level.

Quantization Staircase Validation: When fitting a synthetic quantized line $y = \text{round}(0.35x + 15)$, unpenalized least squares splits the slope into alternating integer steps ($m = 0$ and $m = 1$). Clamping the minimum residual at $\sigma_q^2 = 1/12$ prevents this behavior, recovering $k = 1$ segment with slope $m = 0.3505$ and intercept $c = 14.969$.

3.3 Identifiability Analysis (Pair D)

- Input histogram with support threshold: see `p3/histogram_D.png`.
- Support threshold: ≥ 25 pixels per level to ensure robust regression against noise.
- Supported (constrained) input range: $[(1, 52), (54, 54), (56, 58)]$.
- Unsupported (unconstrained) input range: $[(0, 0), (53, 53), (55, 55), (59, 255)]$.
- Output scatter plot with shaded unconstrained regions: see `p3/scatter_D.png`.
- Segments reported: 2.

Justification of Omission: The sky crop contains almost exclusively dark background pixels. Above intensity level 52, there are zero (or fewer than 25) pixels. The observed data provides no constraints over the higher intensity ranges; any arbitrary segment parameters fit the unsampled domain equally well. Reporting unobserved segments would be unidentifiable extrapolation.

4 Problem 4: Histograms

4.1 Global Equalisation

- Visual output: see `output/p4/equalized_output.png`.
- Histograms and CDF comparison: see `output/p4/equalization_comparison.png`.

(a) Proof: Discrete Equalised Histogram Cannot Be Flat: Let an image contain N pixels and $L = 256$ intensity levels. An ideally flat discrete histogram requires $h(v) = \frac{N}{L}$ pixels for every level $v \in [0, L - 1]$. Pointwise equalization is a deterministic mapping $T : \{0, \dots, L - 1\} \rightarrow \{0, \dots, L - 1\}$. If any input gray level v_k has an initial population $h_{\text{in}}(v_k) > \frac{N}{L}$, all these pixels must map to the same output level $T(v_k)$. Pointwise mapping cannot divide identical input gray levels across multiple output bins:

$$h_{\text{out}}(T(v_k)) \geq h_{\text{in}}(v_k) > \frac{N}{L} \quad (10)$$

Because total probability sums to 1, some output bins are left empty. A flat discrete histogram is therefore impossible unless the input histogram is already uniform.

(b) Proof: Discrete Equalisation is Exactly Idempotent: Let $T_1(v) = \text{round}((L - 1)F_1(v))$. The resulting image has CDF $F_2(z)$. For any output level $z = T_1(v)$, all input pixels with gray levels u mapping to bins $\leq z$ are accumulated. Because T_1 is monotonically non-decreasing, $\{u : T_1(u) \leq T_1(v)\} = \{u : u \leq v_{\text{max}}\}$ where $v_{\text{max}} = \max\{w : T_1(w) = T_1(v)\}$. Thus:

$$F_2(T_1(v)) = F_1(v_{\text{max}}) = \frac{T_1(v)}{L - 1} \quad (11)$$

Applying a second equalization pass:

$$T_2(T_1(v)) = \text{round}((L - 1)F_2(T_1(v))) = \text{round}\left((L - 1)\frac{T_1(v)}{L - 1}\right) = T_1(v) \quad (12)$$

Hence $T_2 \circ T_1 = T_1$, proving exact idempotency.

(c) Invariant Image Construction: Equalization acts as the identity mapping $T(v) = v$ on an image with a uniform histogram (or a linear intensity ramp spanning 0 to 255 with equal pixel counts per level).

4.2 4.2 Histogram Matching and Specification

Target histogram specification computes mapping $T(v) = \arg \min_z |F_{\text{target}}(z) - F_{\text{src}}(v)|$. Wasserstein-1 distance on normalized histograms measures transport cost:

$$W_1(h_1, h_2) = \sum_{v=0}^{255} |F_1(v) - F_2(v)| \quad (13)$$

- Image matching output: see `p4/matching_4_2a.png`.
- Specification output (Uniform and Gaussian $\mu = 128, \sigma = 35$): see `p4/specification_4_2b.png`.

Table 7: Wasserstein-1 distances before and after histogram matching/specification.

Target Case	W_1 Before (levels)	W_1 After (levels)
Matching (<code>reference.png</code>)	40.7746	0.6694
Specification (Uniform Target)	69.1464	1.1857
Specification (Gaussian $\mu = 128, \sigma = 35$)	67.0852	0.4831

Uniform Specification vs Plain Equalisation: Continuous histogram equalisation yields a uniform distribution. In discrete implementations, standard equalisation scales the normalized source CDF directly as $T(v) = \text{round}(255F_{\text{src}}(v))$. Inverse-CDF specification maps $F_{\text{src}}(v)$ to the nearest discrete cumulative quantile of the target uniform histogram via nearest-neighbor search. This can introduce minor boundary variations, but both methods converge to the same monotonic profile.

Colour Equalisation and Hue Preservation: Circular hue distance on HSV pixels ($H \in [0, 1]$, scaled to $[0^\circ, 180^\circ]$) with saturation threshold $S \geq 0.10$:

$$d(h_1, h_2) = \min(|h_1 - h_2|, 1 - |h_1 - h_2|) \times 360^\circ \quad (14)$$

- Output visual comparisons: see `output/p4/colour_4_2c.png`.
- Mean Hue Shift (Per-channel RGB): 60.49° .
- Mean Hue Shift (Luminance-only): 0.38° .

Failure of Independent RGB Equalisation: Color perception depends on the ratios between R, G, and B components. Independent per-channel equalization stretches the color channels by differing amounts, altering these ratios and causing severe hue shifts and oversaturation. Equalizing luminance ($Y = 0.299R + 0.587G + 0.114B$) scales color channels proportionally ($R' = R \frac{Y'}{Y}$), which preserves chromaticity and hue angles.

4.3 4.3 Local Histogram Processing

- Local patch histograms : see `p4/patches_4_3a.png`. Local patches exhibit starkly different multimodal, highlight-clipped, and shadow-skewed distributions, confirming that no single monotonic global curve can simultaneously enhance contrast across all areas.
- Plain AHE vs CLAHE output: see `p4/ahe_vs_clahe_4_3bc.png`.

Plain Tiled AHE Failure Modes:

1. **Boundary Checkerboard Discontinuities:** Independent tile-level equalization produces abrupt step changes in mapping functions across neighboring tile edges.
2. **Noise Over-amplification:** Homogeneous regions (e.g., flat skies) produce narrow histogram spikes. Equalizing these stretches low-level sensor noise across the full dynamic range.

CLAHE Grid Ablation Analysis:

- Output parameter sweep grid: see `p4/ablation_grid_4.3d.png`.
- **Clip Limit Degeneracies:** At $\text{clip} = 1.0$, the histogram is clipped to its mean bin height, limiting contrast expansion and leaving the image nearly unchanged. At $\text{clip} = \infty$, CLAHE reverts to unconstrained AHE, causing strong noise amplification in flat regions.
- **Tile Count Degeneracies:** At 2×2 , tile windows are too large to adapt to local features, approaching global equalization. At 64×64 , tile sizes approach the pixel scale, creating flattened, unnatural contrasts.

Local Histogram Specification: Using a Gaussian target distribution ($\mu = 128, \sigma = 45$) avoids the aggressive midtone stretching produced by uniform CLAHE, maintaining smooth photographic tonal transitions (see `p4/local_spec_4.3e.png`). This requires specifying the target distribution shape parameters (μ, σ) .

4.4 Automatic Exposure Correction

The correction algorithm applies a parameter-free luminance estimator:

1. Compute BT.601 luma $Y = \text{round}(0.299R + 0.587G + 0.114B)$.
 2. Estimate dynamic range percentiles: $v_{\text{low}} = P_1(Y)$ and $v_{\text{high}} = P_{99}(Y)$, normalizing $u = \text{clip}\left(\frac{Y - v_{\text{low}}}{v_{\text{high}} - v_{\text{low}}}, 0, 1\right)$.
 3. Calculate unclipped median $m = \text{median}(u[Y < 255])$ and solve for correction gamma: $\gamma = \frac{\ln(0.5)}{\ln(m)}$.
 4. Compute adjusted luma $Y_{\text{new}} = 255 \cdot u^\gamma$ and scale RGB channels proportionally: $I_{\text{out}} = I_{\text{in}} \cdot \frac{Y_{\text{new}}}{Y}$.
- Corrected exposure set and aligned histograms: see `/p4/exposure_correction_4.4.png`.

Failure Case Analysis: The median-driven exposure estimator fails severely on high-key and overexposed captures (`ev3` and `ev4`). The reason for the failure seems to be as follows:-
Irreversible Highlight Saturation: Severe overexposure clips large regions of the scene (the face and space suit) to the upper sensor ceiling ($Y = 255$), permanently destroying gradient and texture information. Monotonic pointwise tone curves cannot reconstruct lost spatial detail from flat, saturated regions.

Table 8: Pairwise Wasserstein-1 distances between exposure brackets before and after correction.

Image Pair	W_1 Before (levels)	W_1 After (levels)
ev0 $\leftrightarrow$ ev1	28.9065	0.3617
ev0 $\leftrightarrow$ ev2	86.6320	0.4178
ev0 $\leftrightarrow$ ev3	143.0182	57.1984
ev0 $\leftrightarrow$ ev4	165.3108	80.8509
ev1 $\leftrightarrow$ ev2	57.7256	0.2518
ev1 $\leftrightarrow$ ev3	114.1118	56.8942
ev1 $\leftrightarrow$ ev4	136.4043	80.5467
ev2 $\leftrightarrow$ ev3	56.3862	56.8667
ev2 $\leftrightarrow$ ev4	78.6788	80.5193
ev3 $\leftrightarrow$ ev4	22.2926	23.6525
Mean Pairwise W_1	88.9467	43.7560
Max Pairwise W_1	165.3108	80.8509

Table 9: Output luminance distribution metrics.

Corrected Image	Luma Standard Deviation (σ)	1st–99th Percentile Range
ev0	76.92	253.00
ev1	76.89	253.00
ev2	76.75	254.00
ev3	96.89	255.00
ev4	93.77	255.00

5 Problem 5: Green Screen Compositing

5.1 Footage Selection and Justification

A 720p sequence (1280×720 at 24 fps) was extracted into 48 sequential frames (f000.png–f047.png). The clip features a subject against an illuminated green backdrop, characterized by fine flyaway hair strands, an earphone cord, and edge motion blur.

5.2 5.1 Naive Key

- Output failure visualization: see `output/p5/p5.1_naive_failure.png`.
- Estimated key backing colour: $(C_b, C_r) = (42.0, 86.0)$ computed over a 24-pixel perimeter border.

Table 10: Example pixel that got misclassified as foreground ($Y = 419, X = 801$).

Metric / Space	Evaluated Value	Classification Verdict
Raw RGB	$R = 44, G = 102, B = 0$	—
Green Excess Metric	$G - \max(R, B) = 58$	$\alpha = 1.0$ (Mistaken Foreground, ≤ 60)
BT.601 YCbCr	$Y = 73.0, C_b = 87.0, C_r = 107.0$	—
Chroma Distance (d)	49.66	$\alpha = 0.0$ (Correct Backing, $< t_{\text{out}}$)

Failure Mechanism of Naive RGB Dominance: Naive RGB keying evaluates green dominance via $\Delta G = G - \max(R, B) > \tau$. Because R, G, B intensities scale proportionally with local irradiance E , shadow regions attenuate absolute channel values:

At pixel (419, 801), irradiance ($\gamma \approx 0.55$), dropped green excess to below the threshold of 60 and mistakenly classifying backing as solid foreground. In contrast, BT.601 decouples luminance Y from chromaticity:

$$\begin{bmatrix} C_b \\ C_r \end{bmatrix} = \begin{bmatrix} -0.168736 & -0.331264 & 0.500000 \\ 0.500000 & -0.418688 & -0.081312 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + 128 \quad (15)$$

In this euclidean distance $d = \sqrt{(C_b - k_{Cb})^2 + (C_r - k_{Cr})^2}$ is invariant to intensity. Therefore, it correctly identified the pixel as background.

Backing Uniformity and Severity of Failure: Under studio lighting, naive RGB failure is mild and localized to the subject’s edges and the audio wire. However, in non-uniformly illuminated backings, irradiance γ can vary across the canvas. In such condition a single global threshold value fails catastrophically: requiring a need for such a classification.

5.3 5.2 Soft Matte and Spill Suppression

- Output chroma distance distributions: see `p5/p5_2a_chroma_distributions.png`.
- Composites with and without spill suppression: see `p5/p5_2c_spill_comparison.png`.
- Matte comparison and absolute difference field: see `output/p5/p5_2d_matte_comparison.png`.

(a) Chroma Distance Population Separation: Chroma distances d from key colour (42.0, 86.0) reveal two dominant modes: pure backing clustered at $d \in [0, 15]$ and solid foreground (skin and clothing) centered at $d \in [80, 115]$. The populations do not separate cleanly. Partially transparent pixels form an intermediate range spanning $d \in [15, 80]$. Therefore, a single binary threshold is mathematically incapable of segmenting the image. Setting a low threshold clips semi-transparent hair fibers, creating unnatural jagged lines; setting a high threshold retains an opaque green background. Thus, a continuous ramp between $[t_{in}, t_{out}]$ is necessary.

(b) Continuous Alpha Matte Generation: Alpha values are computed via continuous piecewise-linear normalization:

$$\alpha(d) = \text{clip} \left(\frac{d - t_{in}}{t_{out} - t_{in}}, 0.0, 1.0 \right), \quad t_{in} = [15.0], \quad t_{out} = [80.0] \quad (16)$$

The composite image is synthesized via linear alpha blending: $I_{comp} = \alpha F + (1 - \alpha)B$.

- **Fraction of Fractional Alpha Pixels** ($0.05 < \alpha < 0.95$): 0.50%.

(c) Spill Suppression and Residual Green Excess: Optical scattering and transmission mix green backing light into translucent foreground boundaries. The despill operator clamps excessive green channel values to the average of red and blue:

$$G_{clean} = G - (1 - \alpha) \cdot \max \left(0, G - \frac{R + B}{2} \right) \quad (17)$$

Justification of $(1 - \alpha)$ Scaling: Opaque foreground regions ($\alpha = 1$) require no chromatic alteration ($1 - \alpha = 0$), preserving natural clothing or olive skin pigments. Completely transparent

Table 11: Spill suppression evaluation over fractional alpha pixels ($0.05 < \alpha < 0.95$).

Metric	Value
Mean Green Excess Before Despill	[82.60]
Mean Green Excess After Despill	[30.68]
Spill Reduction Efficiency	[62.9%]

background pixels ($\alpha = 0$) are fully replaced by the background plate. Spill attenuation must scale proportionally to backing exposure ($1 - \alpha$), targeting boundary pixels where green contamination is present.

Importance of Fractional Mask Restriction: Evaluating green excess across the entire image skews the metric: solid clothing can contain natural yellow-green components that register as false spill, while pure backing pixels exhibit high excess that dominates the mean. Restricting evaluation strictly to $0.05 < \alpha < 0.95$ isolates optical boundary mixtures where despill is functionally active.

(d) Matte Difference and Disagreement Analysis: Comparing the naive binary matte (α_{naive}) against the continuous soft matte (α_{soft}):

$$\text{MAD} = \frac{1}{HW} \sum_{x,y} |\alpha_{\text{soft}} - \alpha_{\text{naive}}|, \quad \text{GradError} = \frac{1}{HW} \sum_{x,y} \|\nabla \alpha_{\text{soft}} - \nabla \alpha_{\text{naive}}\|_2 \quad (18)$$

Table 12: Quantitative disagreement metrics between naive binary and continuous soft mattes.

Metric	Value
Mean Absolute Difference (MAD)	[0.0014]
Starter Scaled SAD	[1.30]
Gradient Field Error	[0.67]

Reason for Sharp Disagreement: MAD computes a zero-order spatial average across the entire frame. Because fractional boundary pixels occupy a small percentage of total canvas area, MAD remains near zero ([0.0014]), concealing boundary discrepancies. Conversely, the gradient field error evaluates first-order spatial derivatives: naive thresholding introduces step-function discontinuities ($|\nabla \alpha_{\text{naive}}| \rightarrow \infty$ across boundaries), whereas the soft matte exhibits bounded, continuous transition slopes.

Visual Predictor: Gradient Field Error accurately predicts perceptual quality in the composite. Human visual perception (via Mach banding and lateral inhibition) is sensitive to high-frequency edge discontinuities and hard halos. A small MAD can still yield visible composite seams, whereas minimizing gradient field error ensures smooth boundary integration.

5.4 5.3 Video, Efficiently

The 48 frames were keyed using a 128-bin ChromaLUT and composited against a 3200×720 panning background plate ($2.50 \times$ frame width, 1920 px shift). The resulting video was encoded to `output/p5/p5_composite.mp4`.

Bottleneck and Throughput: Overall throughput reached **2.86 FPS** (16.79 s total). The primary bottleneck is **Spill Suppression** (102.60 ms, 30.0%), driven by per-pixel 3-channel conditional clamping and arithmetic across 921,600 pixels.

Table 13: Per-frame latency breakdown.

Pipeline Stage	Latency (ms)	Share (%)
Disk I/O (Read PNGs)	16.34	4.8%
Vectorized LUT Keying	89.45	26.1%
Spill Suppression	102.60	30.0%
Alpha Blending Compositing	96.74	28.3%
Video Encoding (x264)	37.17	10.9%

Temporal Flicker and Edge Fidelity: Full-frame differences conflate gross subject motion with sensor noise; thus, flicker was evaluated over a static boundary ROI. An Exponential Moving Average filter ($\tilde{\alpha}_t = 0.70\alpha_t + 0.30\tilde{\alpha}_{t-1}$) reduced mean frame difference from **0.000444** to **0.000399** (**10.1% reduction**).

Cross-sectional edge profiles (see `output/p5/p5_3_flicker_and_lag.png`) confirm that the boundary slope remains steep without spatial smearing, and tracks the frame-to-frame boundary shift with zero residual trailing or motion lag.